Aula 6 - Sobrecarga de Métodos e Tratamento de exceção

Programação Orientada a Objetos
Prof. Matheus Avila Moreira de Paula
matheusavila@cefetmg.br

Engenharia de Computação
Centro Federal de Educação Tecnológica Leopoldina

Sobrecarga de Métodos

- Para algumas classes é desejável possuir mais de uma opção para a instanciação de um objeto. Ou seja, criar mais de um construtor.
- Podemos fazer isso através da **sobrecarga de métodos**

Sobrecarga de Métodos

```
public static void soma(int operando1, int operando2){
    int result =  operando1 + operando2;
    System.out.println("soma = " + result);
}

public static void soma(double operando1, double operando2){
    double result =  operando1 + operando2;
    System.out.println("soma = " + result);
}
```

Exercícios

- Crie uma classe chamada estudante que possui como atributos: nome, matrícula, número de disciplinas, idade, sexo.
  - Crie um construtor que inicia o nome, sexo, idade e matrícula
  - Crie um construtor que inicia o nome e matrícula

Tratamento de Exceção

- Uma Exceção é uma indicação de um problema que ocorreu em alguma linha durante a execução de um programa. Representa uma situação que não pôde ser prevista durante a compilação mas levou a uma situação incomum
  - Por exemplo:
    - Leitura de um arquivo inexistente
    - Aplicação de algum operador em variáveis lidas do teclado
    - Erros numéricos (divisão por 0, onde 0 é o valor de uma variável lida pelo teclado)

Tratamento de Exceção

- Tratamentos de exceções visam contornar esses erros através de testes.
  - Ao invés de encerrar o programa por não ter realizado a leitura de um arquivo, podemos **capturar** esse erro é tratar para impedir o término do programa

Tratamento de Exceção

- O método main abaixo chama a função soma passando 2 Strings como parâmetro

```
System.out.println("Resultado: " + soma("12", "13"));
```

- A função soma recebe qualquer tipo como parâmetro. Entretanto, isso não significa que ela irá conseguir realizar as mesmas operações com todos os tipos dados

```
public static <T> double soma(T op1, T op2){
    double res;
    res = (double) op1 + (double) op2;
    System.out.println(res);
    return res;
}
```

Tratamento de Exceção

- Ao tentarmos executar o código nos deparamos com o erro:

```
Exception in thread "main" java.lang.ClassCastException: class java.lang.String cannot be cast to class java.lang.Double
```

- O nome da exceção indicado é ClassCastException
- Esta exceção ocorreu devido a incapacidade de converter os parâmetros: uma String para double

Tratamento de Exceção

- Para evitar a interrupção do programa podemos colocar o ponto de execução que pode gerar erros dentro de um bloco try

```
public static <T> double soma(T op1, T op2){
    double res;
    try{
    res = (double) op1 + (double) op2;}
    catch(ClassCastException e){
        res = -1;
    }
    System.out.println(res);
    return res;
}
```

- Dessa forma, quando um erro ocorrer dentro de um bloco try ele será tratado por um bloco catch()

Tratamento de Exceção

- Podemos tratar esse erro de outra forma. Já que sabemos que o erro se deve a conversão incorreta de uma String para double podemos fazer a seguinte correção:

```
public static <T> double soma(T op1, T op2){
    double res;
    try{
    res = (double) op1 + (double) op2;
    }
    catch(Exception e){
    res = Double.parseDouble(op1.toString()) + Double.parseDouble(op2.toString());
    }

    System.out.println(res);
    return res;
}
```

Tratamento de Exceção

```
System.out.println("Resultado: " + soma(4.1, 6.0));
System.out.println("Resultado: " + soma("12", "13"));
```

- Na primeira chamada a operação é concluída com sucesso

```
public static <T> double soma(T op1, T op2){
    double res = -1;
    try{
    res = (double) op1 + (double) op2;
    BufferedWriter writer = new BufferedWriter(new FileWriter("pasta/teste.txt"));
    writer.write(Double.toString(res));
    writer.close();
    }
    catch(IOException e){
        System.out.println("deu ruim na escrita!");
    }
    catch(ClassCastException e){
        System.out.println("Erro na soma");
    res = Double.parseDouble(op1.toString()) + Double.parseDouble(op2.toString());
    }

    System.out.println(res);
    return res;
}
```

```
10.1
Resultado: 10.1
Erro na soma
25.0
Resultado: 25.0
```

Tratamento de Exceção

```
System.out.println("Resultado: " + soma(4.1, 6.0));
System.out.println("Resultado: " + soma("12", "13"));
```

- Na segunda chamada a operação de transformação em double falha (ClassCastException) e é tratada pelo segundo catch

```
public static <T> double soma(T op1, T op2){
    double res = -1;
    try{
    res = (double) op1 + (double) op2;
    BufferedWriter writer = new BufferedWriter(new FileWriter("pasta/teste.txt"));
    writer.write(Double.toString(res));
    writer.close();
    }
    catch(IOException e){
        System.out.println("deu ruim na escrita!");
    }
    catch(ClassCastException e){
        System.out.println("Erro na soma");
    res = Double.parseDouble(op1.toString()) + Double.parseDouble(op2.toString());
    }

    System.out.println(res);
    return res;
}
```

```
10.1
Resultado: 10.1
Erro na soma
25.0
Resultado: 25.0
```

Tratamento de Exceção

```
System.out.println("Resultado: " + soma(4.1, 6.0));
System.out.println("Resultado: " + soma("12", "13"));
```

- Na primeira chamada, a operação de escrita em arquivo falha pois agora, temos o nome do diretório escrito incorretamente.

```
public static <T> double soma(T op1, T op2){
    double res = -1;
    try{
    res = (double) op1 + (double) op2;
    BufferedWriter writer = new BufferedWriter(new FileWriter("pasa/teste.txt"));
    writer.write(Double.toString(res));
    writer.close();
    }
    catch(IOException e){
        System.out.println("deu ruim na escrita!");
    }
    catch(ClassCastException e){
        System.out.println("Erro na soma");
    res = Double.parseDouble(op1.toString()) + Double.parseDouble(op2.toString());
    }

    System.out.println(res);
    return res;
}
```

```
deu ruim na escrita!
10.1
Resultado: 10.1
Erro na soma
25.0
Resultado: 25.0
```

Exercícios

- Faça uma função genérica que faça a divisão de dois valores passados como parâmetro. Sua função deverá tratar as exceções de divisão por zero e divisão de Strings. Caso a String não possa ser convertida para número, apresente uma mensagem de erro e retorne -1 na função.
- Faça um menu, utilizando *switch-case,* onde as opções são: 0- sair, 1- imprime “Hello World” e 2- imprime “Voce digitou 2” e trate o caso em que o usuário escreve um caractere ao invés de um valor inteiro.